Q1. Missing Data Identification

Scenario: The hospital suspects incomplete patient records.

Task:

Identify missing values in each column

Calculate percentage of missing data

In [1]:
```
import pandas as pd
```

In [3]:
```
df = pd.read_csv('healthcare_data_cleaning_dataset.csv')
```

In [4]:
```
missing_values = df.isnull().sum()
```

In [5]:
```
missing_percentage = (df.isnull().sum() / len(df)) * 100
```

In [6]:
```
print("Missing Values:")
print(missing_values)
```

```
Missing Values:
Patient_ID             0
Age                  600
Gender                 0
City                   0
Diagnosis              0
Hospital_Visits        0
Treatment_Cost       593
Insurance_Coverage     0
Admission_Date         0
dtype: int64
```

In [7]:
```
print("Missing Percentage:")
print(missing_percentage)
```

```
Missing Percentage:
Patient_ID            0.000000
Age                  11.764706
Gender                0.000000
City                  0.000000
Diagnosis             0.000000
Hospital_Visits       0.000000
Treatment_Cost       11.627451
Insurance_Coverage    0.000000
Admission_Date        0.000000
dtype: float64
```

Q2. Handling Missing Age

Scenario: Age is critical for medical analysis, but some values are missing.

Task:

Replace missing Age values with an appropriate method

Justify your choice (mean/median)

In [9]:
```
df['Age'] = df['Age'].fillna(df['Age'].median())
```

Median is used because it is less affected by outliers and gives a better central value for Age data.

Q3. Handling Missing Treatment Cost

Scenario: Treatment cost is highly skewed due to expensive treatments.

Task:

Handle missing Treatment_Cost values

Choose the correct imputation method and explain why

In [10]:
```
df['Treatment_Cost'] = df['Treatment_Cost'].fillna(df['Treatment_Cost'].median())
```

Median is used because Treatment_Cost is highly skewed, and median is not affected by extreme values (outliers).

Q4. Duplicate Patient Records

Scenario: Some patient records were entered multiple times.

Task:

Identify duplicate rows

Remove duplicates

Compare dataset size before and after

In [11]:
```
duplicates = df.duplicated().sum()
print("Number of duplicate rows:", duplicates)
```

```
Number of duplicate rows: 99
```

In [12]:
```
before_shape = df.shape
print("Before removing duplicates:", before_shape)
```

```
Before removing duplicates: (5100, 9)
```

In [13]:
```
df = df.drop_duplicates()
```

In [14]:
```
after_shape = df.shape
print("After removing duplicates:", after_shape)
```

```
After removing duplicates: (5001, 9)
```

Q5. Invalid Age Values (Data Quality Check)

Scenario: Some patients have unrealistic age values (e.g., >100 or <0).

Task:

Detect such records

Decide whether to remove or correct them

In [15]:
```
invalid_age = df[(df['Age'] < 0) | (df['Age'] > 100)]
```

In [16]:
```
print("Invalid Age Records:")
print(invalid_age)
```

```
Invalid Age Records:
Empty DataFrame
Columns: [Patient_ID, Age, Gender, City, Diagnosis, Hospital_Visits, Treatment_Cost, Insurance_Coverage, Admission_Date]
Index: []
```

In [17]:
```
df = df[(df['Age'] >= 0) & (df['Age'] <= 100)]
```

These values are unrealistic and likely data entry errors, so removing them improves data quality.

Q6. Outlier Detection (Treatment Cost)

Scenario: Extreme treatment costs are affecting analysis.

Task:

Detect outliers using IQR method

Display number of outliers

In [18]:
```
Q1 = df['Treatment_Cost'].quantile(0.25)
Q3 = df['Treatment_Cost'].quantile(0.75)
```

In [19]:
```
IQR = Q3 - Q1
```

In [20]:
```
outliers = df[(df['Treatment_Cost'] < (Q1 - 1.5 * IQR)) |
              (df['Treatment_Cost'] > (Q3 + 1.5 * IQR))]
```

In [21]:
```
print("Number of outliers:")
print(outliers.shape[0])
```

```
Number of outliers:
50
```

Q7. Outlier Treatment

Scenario: The business team wants to retain all records.

Task:

Apply capping (Winsorization) on Treatment_Cost

Use 5th and 95th percentile

In [22]:
```
lower_limit = df['Treatment_Cost'].quantile(0.05)
upper_limit = df['Treatment_Cost'].quantile(0.95)
```

In [23]:
```
df['Treatment_Cost'] = df['Treatment_Cost'].clip(lower=lower_limit, upper=upper_limit)
```

In [25]:
```
print(df['Treatment_Cost'])
```

```
0       41010.0
1       12194.0
2       45086.0
3       40842.0
4        9873.0
         ...
4996    24797.0
4997     3238.0
4998    31337.0
4999    39801.0
5095    24797.0
Name: Treatment_Cost, Length: 5001, dtype: float64
```

Q8. Transformation

Scenario: Treatment cost is highly skewed.

Task:

Apply log transformation

Create a new column

Compare before vs after distribution

In [27]:
```
import numpy as np
```

In [28]:
```
df['Log_Treatment_Cost'] = np.log1p(df['Treatment_Cost'])
```

In [29]:
```
print("Before Transformation:")
print(df['Treatment_Cost'].describe())
```

```
Before Transformation:
count     5001.000000
mean     25218.878624
std      13580.445490
min       3238.000000
25%      13766.000000
50%      24797.000000
75%      36542.000000
max      47948.000000
Name: Treatment_Cost, dtype: float64
```

In [30]:
```
print("After Transformation:")
print(df['Log_Treatment_Cost'].describe())
```

```
After Transformation:
count    5001.000000
mean        9.918335
std         0.751774
min         8.083020
25%         9.530030
50%        10.118518
75%        10.506245
max        10.777893
Name: Log_Treatment_Cost, dtype: float64
```

Q9. Time-Based Missing Handling

Scenario: Admission dates should follow a logical sequence.

Task:

Sort data by Admission_Date

Apply forward fill or backward fill where appropriate

Justify your choice

In [31]:
```
df['Admission_Date'] = pd.to_datetime(df['Admission_Date'])
```

In [32]:
```
df = df.sort_values(by='Admission_Date')
```

In [34]:
```
df = df.ffill()
```

Forward fill is used because in time-based data, the previous valid value is usually the most logical replacement for missing values.